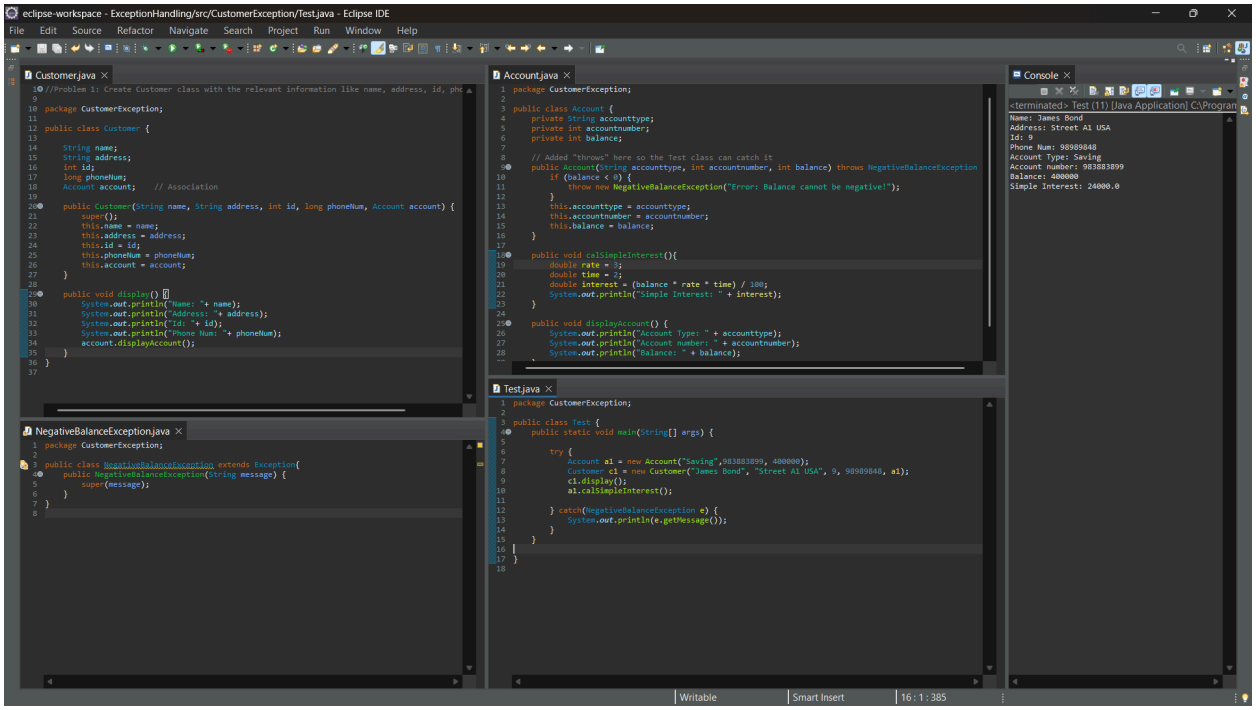
Name: Sanskar Soni
Subject: JAVA
Assignment: 9
Form Number: 251101107

**Problem 1: Create Customer class with the relevant information like name, address, id, phone
no etc. Write a parameterized constructor and relevant methods (disp(), etc) appropriately.
Create Account class with account type, account number, minimum balance etc., Write calculateInterest method (use simple interest – assume time and rate appropriately). Create a
user defined exception class "NegativeBalanceException" and throw that exception when
there is negative balance while calculating the interest.
Use Account class in Customer class display the details of customer with account information.
(No Inheritance – use association only)**

Solution:

```
package CustomerException;
public class Customer {
	String name;
	String address;
	int id;
	long phoneNum;
	Account account;  // Association

	public Customer(String name, String address, int id, long phoneNum, Account account) {
		super();
		this.name = name;
		this.address = address;
		this.id = id;
		this.phoneNum = phoneNum;
		this.account = account;
	}

	public void display() {
		System.out.println("Name: "+ name);
		System.out.println("Address: "+ address);
		System.out.println("Id: "+ id);
		System.out.println("Phone Num: "+ phoneNum);
		account.displayAccount();
	}
}
package CustomerException;
public class Account {
  private String accounttype;
  private int accountnumber;
```

```java
    private int balance;
    // Added "throws" here so the Test class can catch it
    public Account(String accounttype, int accountnumber, int balance) throws NegativeBalanceException {
        if (balance < 0) {
            throw new NegativeBalanceException("Error: Balance cannot be negative!");
        }
        this.accounttype = accounttype;
        this.accountnumber = accountnumber;
        this.balance = balance;
    }
    public void calSimpleInterest(){
        double rate = 3;
        double time = 2;
        double interest = (balance * rate * time) / 100;
        System.out.println("Simple Interest: " + interest);
    }
    public void displayAccount() {
        System.out.println("Account Type: " + accounttype);
        System.out.println("Account number: " + accountnumber);
        System.out.println("Balance: " + balance);
    }
}
package CustomerException;
public class NegativeBalanceException extends Exception{
        public NegativeBalanceException(String message) {
                super(message);
        }
}
package CustomerException;
public class Test {
        public static void main(String[] args) {

                try {
                        Account a1 = new Account("Saving",983883899, 400000);
                        Customer c1 = new Customer("James Bond", "Street A1 USA", 9, 98989848, a1);
                        c1.display();
                        a1.calSimpleInterest();

                } catch(NegativeBalanceException e) {
                        System.out.println(e.getMessage());
                }
        }
}
```

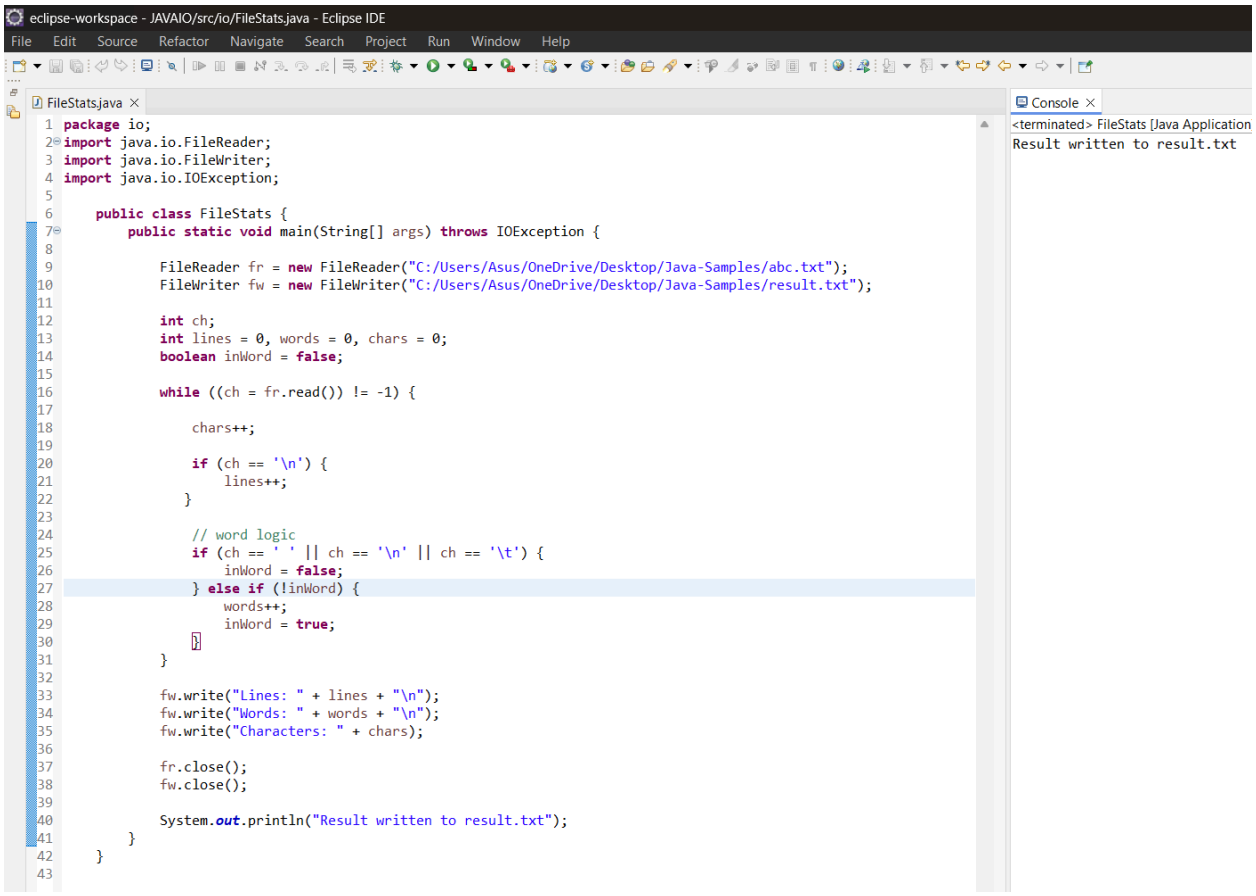


1. Write a Program to read the same program file and find the no. of lines, words and characters. Write the result in in to a text file (result.txt)

Solution:

```
package io;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
        public class FileStats {
            public static void main(String[] args) throws IOException {
                FileReader fr = new
FileReader("C:/Users/Asus/OneDrive/Desktop/Java-Samples/abc.txt");
                FileWriter fw = new
FileWriter("C:/Users/Asus/OneDrive/Desktop/Java-Samples/result.txt");
                int ch;
                int lines = 0, words = 0, chars = 0;
                boolean inWord = false;
                while ((ch = fr.read()) != -1) {
                    chars++;
                    if (ch == '\n') {
                        lines++;
```

```java
            }
            // word logic
            if (ch == ' ' || ch == '\n' || ch == '\t') {
                inWord = false;
            } else if (!inWord) {
                words++;
                inWord = true;
            }
        }
        fw.write("Lines: " + lines + "\n");
        fw.write("Words: " + words + "\n");
        fw.write("Characters: " + chars);
        fr.close();
        fw.close();
        System.out.println("Result written to result.txt");
    }
}
```

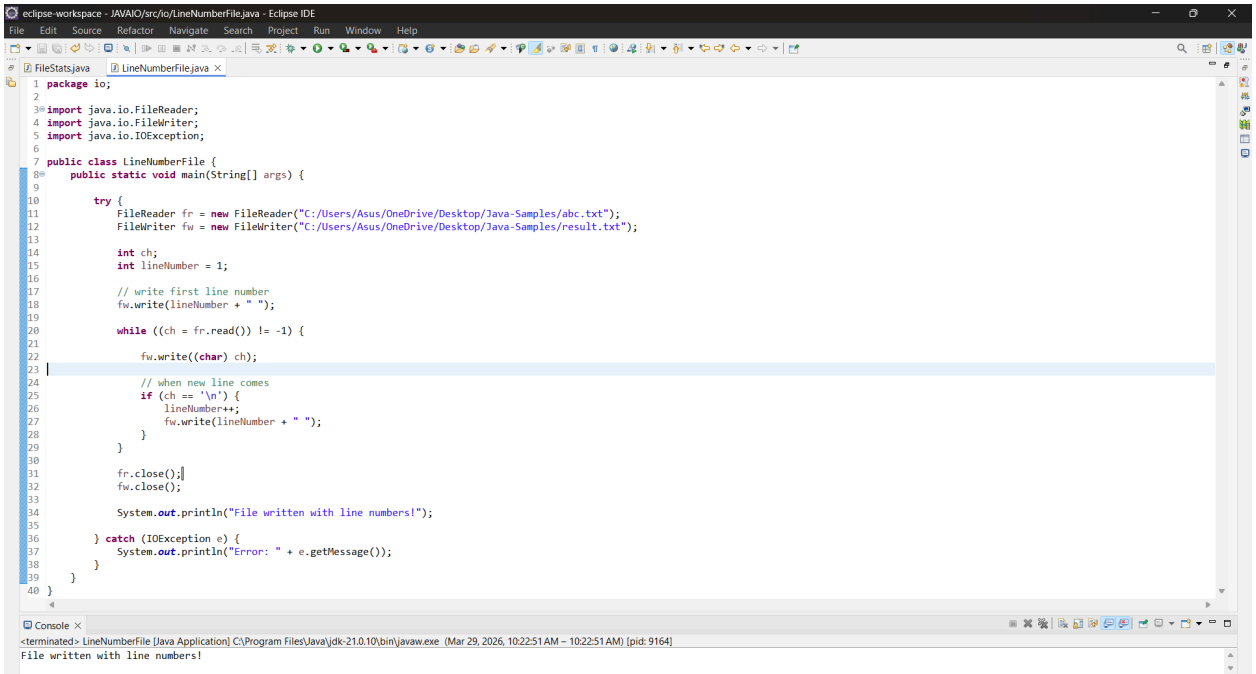


2. Write a program to read the same program file and write it to other file with

the lines number added before each line, starting from 1.

Solution:

```
package io;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class LineNumberFile {
  public static void main(String[] args) {
    try {
      FileReader fr = new FileReader("C:/Users/Asus/OneDrive/Desktop/Java-Samples/abc.txt");
      FileWriter fw = new FileWriter("C:/Users/Asus/OneDrive/Desktop/Java-Samples/result.txt");
      int ch;
      int lineNumber = 1;
      // write first line number
      fw.write(lineNumber + " ");
      while ((ch = fr.read()) != -1) {
        fw.write((char) ch);
        // when new line comes
        if (ch == '\n') {
          lineNumber++;
          fw.write(lineNumber + " ");
        }
      }
      fr.close();
      fw.close();
      System.out.println("File written with line numbers!");
    } catch (IOException e) {
      System.out.println("Error: " + e.getMessage());
    }
  }
}
```

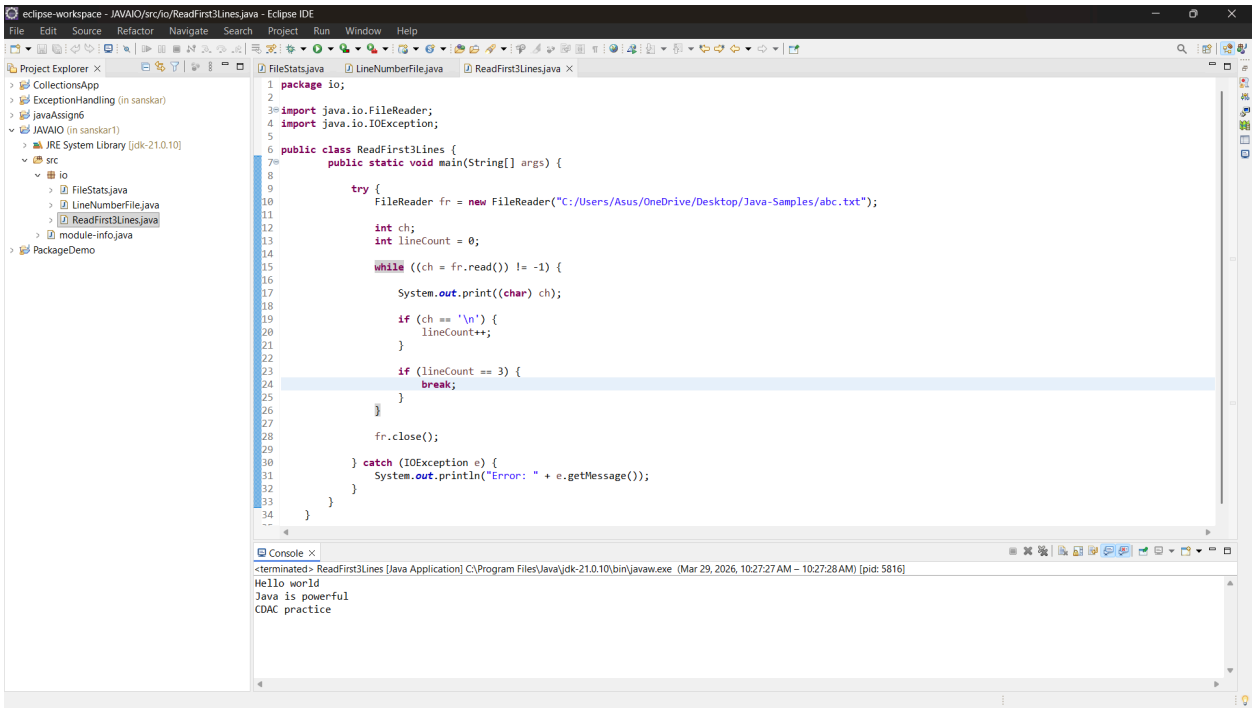


3. Write a Java program to read first 3 lines from a file.

Solution:

```
package io;
import java.io.FileReader;
import java.io.IOException;
public class ReadFirst3Lines {
        public static void main(String[] args) {
            try {
                FileReader fr = new
FileReader("C:/Users/Asus/OneDrive/Desktop/Java-Samples/abc.txt");
                int ch;
                int lineCount = 0;
                while ((ch = fr.read()) != -1) {
                    System.out.print((char) ch);
                    if (ch == '\n') {
                        lineCount++;
                    }
                    if (lineCount == 3) {
                        break;
                    }
                }
                fr.close();
            } catch (IOException e) {
                System.out.println("Error: " + e.getMessage());
```

```
        }
    }
}
```



4. Write a Java program to find the longest word in a text file

Solution:

```
package io;
import java.io.FileReader;
import java.io.IOException;
public class LongestWord {
        public static void main(String[] args) {
            try {
                FileReader fr = new FileReader("C:/Users/Asus/OneDrive/Desktop/Java-Samples/abc.txt");
                int ch;
                String currentWord = "";
                String longestWord = "";
                while ((ch = fr.read()) != -1) {
                    char c = (char) ch;
                    // if character is not space/newline → build word
                    if (c != ' ' && c != '\n' && c != '\t') {
                        currentWord += c;
                    } else {
                        // word ended → compare
                        if (currentWord.length() > longestWord.length()) {
                            longestWord = currentWord;
```

```
                }
                currentWord = ""; // reset
            }
        }
        // check last word (important)
        if (currentWord.length() > longestWord.length()) {
            longestWord = currentWord;
        }
        fr.close();
        System.out.println("Longest Word: " + longestWord);
    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
  }
}
```

ava - Eclipse IDE

arch Project Run Window Help

FileStats.java | LineNumberFile.java | ReadFirst3Lines.java | LongestWord.java

```java
package io;

import java.io.FileReader;
import java.io.IOException;

public class LongestWord {
        public static void main(String[] args) {

            try {
                FileReader fr = new FileReader("C:/Users/Asus/OneDrive/Desktop/Java-Samples/abc.txt");

                int ch;
                String currentWord = "";
                String longestWord = "";

                while ((ch = fr.read()) != -1) {
                    char c = (char) ch;

                    // if character is not space/newline → build word
                    if (c != ' ' && c != '\n' && c != '\t') {
                        currentWord += c;
                    } else {
                        // word ended → compare
                        if (currentWord.length() > longestWord.length()) {
                            longestWord = currentWord;
                        }
                        currentWord = ""; // reset
                    }
                }

                // check last word (important)
                if (currentWord.length() > longestWord.length()) {
                    longestWord = currentWord;
                }

                fr.close();

                System.out.println("Longest Word: " + longestWord);

            } catch (IOException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
```

Console

```
<terminated> LongestWord [Java Application] C:\Program Files\Java\jdk-21.0.10\bin\javaw.exe  (Mar 29, 2026, 10:3
Longest Word: well-developed,
```

5. Write a programs to implemnt Caeser cipher using files.

Write to the file (enc_mssage.txt) with using caeser cipher with

the displacement value = 3.

Read the file (enc_message.txt) and decode the Cipher text and write it into

a file (dec_message.txt)

Solution:

```
package io;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class CaesarCipherFile {
  public static void main(String[] args) {
    String inputPath = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/abcd.txt";
    String encPath   = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/enc_message.txt";
    String decPath   = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/dec_message.txt";
    try {
      FileReader fr = new FileReader(inputPath);
      FileWriter fw = new FileWriter(encPath);
      int ch;
      while ((ch = fr.read()) != -1) {
        char c = (char) ch;
        if (Character.isLetter(c)) {
          if (Character.isLowerCase(c)) {
            c = (char) ((c - 'a' + 3) % 26 + 'a');
          } else {
            c = (char) ((c - 'A' + 3) % 26 + 'A');
          }
        }
        fw.write(c);
      }
      fr.close();
      fw.close();
      System.out.println("Encryption done! File saved at:");
      System.out.println(encPath);
    } catch (IOException e) {
      System.out.println("Error: " + e.getMessage());
    }
    try {
      FileReader fr = new FileReader(encPath);
      FileWriter fw = new FileWriter(decPath);
      int ch;
      while ((ch = fr.read()) != -1) {
        char c = (char) ch;
        if (Character.isLetter(c)) {
          if (Character.isLowerCase(c)) {
            c = (char) ((c - 'a' - 3 + 26) % 26 + 'a');
          } else {
```

```
                c = (char) ((c - 'A' - 3 + 26) % 26 + 'A');
            }
        }
        fw.write(c);
    }
    fr.close();
    fw.close();
    System.out.println("Decryption done! File saved at:");
    System.out.println(decPath);
  } catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
  }
}
}
```

herFile.java - Eclipse IDE

Search Project Run Window Help

FileStats.java | LineNumberFile.java | ReadFirst3Lines.java | LongestWord.java

```java
package io;

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CaesarCipherFile {

    public static void main(String[] args) {

        String inputPath = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/abcd.txt";
        String encPath   = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/enc_message.txt";
        String decPath   = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/dec_message.txt";

        try {
            FileReader fr = new FileReader(inputPath);
            FileWriter fw = new FileWriter(encPath);

            int ch;

            while ((ch = fr.read()) != -1) {
                char c = (char) ch;

                if (Character.isLetter(c)) {
                    if (Character.isLowerCase(c)) {
                        c = (char) ((c - 'a' + 3) % 26 + 'a');
                    } else {
                        c = (char) ((c - 'A' + 3) % 26 + 'A');
                    }
                }

                fw.write(c);
            }

            fr.close();
            fw.close();

            System.out.println("Encryption done! File saved at:");
            System.out.println(encPath);

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            FileReader fr = new FileReader(encPath);
            FileWriter fw = new FileWriter(decPath);

            int ch;

            while ((ch = fr.read()) != -1) {
                char c = (char) ch;

                if (Character.isLetter(c)) {
                    if (Character.isLowerCase(c)) {
                        c = (char) ((c - 'a' - 3 + 26) % 26 + 'a');
                    } else {
                        c = (char) ((c - 'A' - 3 + 26) % 26 + 'A');
                    }
                }

                fw.write(c);
            }

            fr.close();
            fw.close();

            System.out.println("Decryption done! File saved at:");
            System.out.println(decPath);

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Console

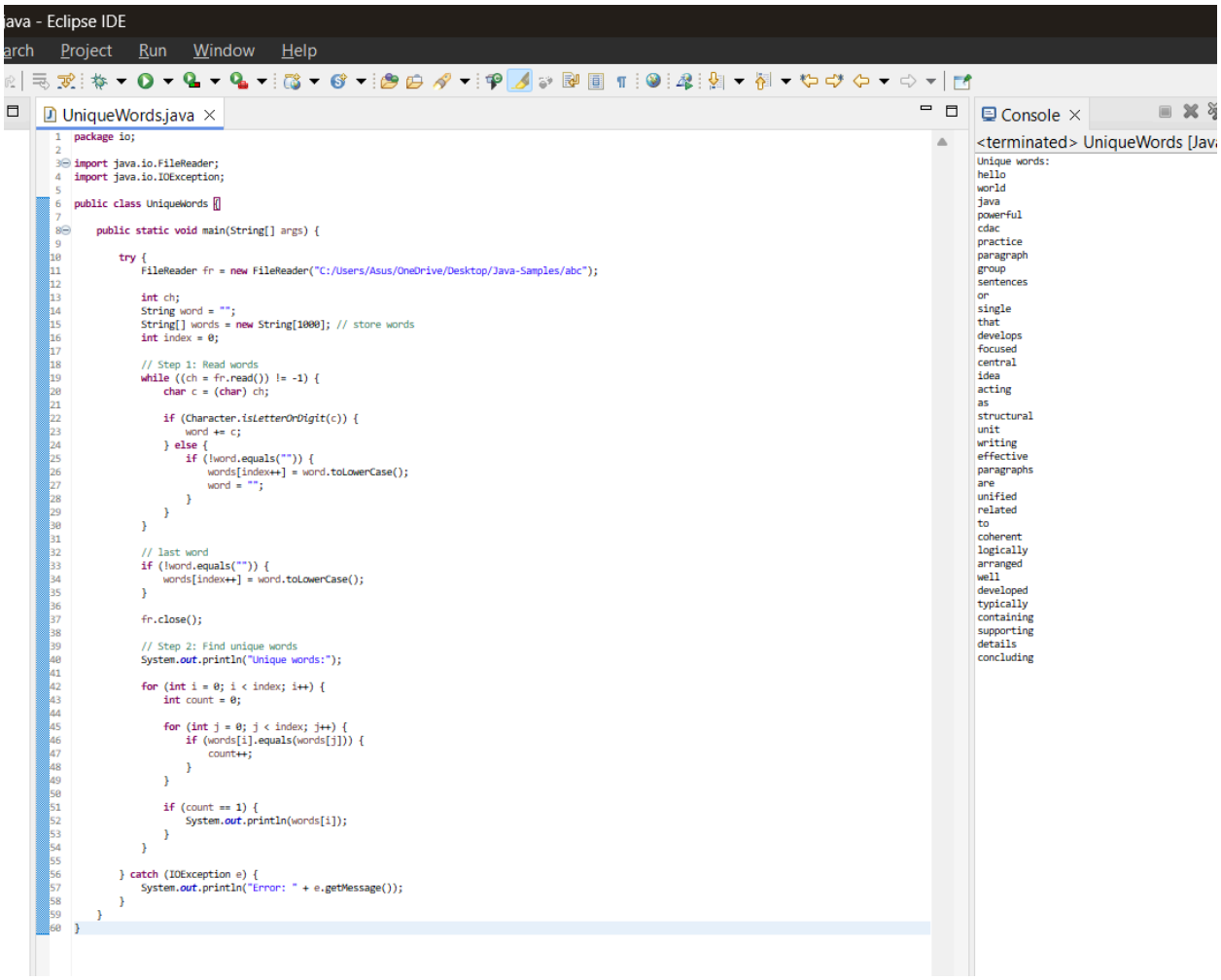
<terminated> CaesarCipherFile [Java Application] C:\Program Files\Java\jdk-21.0.10\bin\javaw.exe (M

```
Encryption done! File saved at:
C:/Users/Asus/OneDrive/Desktop/Java-Samples/enc_message.txt
Decryption done! File saved at:
C:/Users/Asus/OneDrive/Desktop/Java-Samples/dec_message.txt
```

6. Write a program to find unique words in file

Solution:

```
package io;
import java.io.FileReader;
import java.io.IOException;
public class UniqueWords {
  public static void main(String[] args) {
    try {
      FileReader fr = new FileReader("C:/Users/Asus/OneDrive/Desktop/Java-Samples/abc");
      int ch;
      String word = "";
      String[] words = new String[1000]; // store words
      int index = 0;
      // Step 1: Read words
      while ((ch = fr.read()) != -1) {
        char c = (char) ch;
        if (Character.isLetterOrDigit(c)) {
          word += c;
        } else {
          if (!word.equals("")) {
            words[index++] = word.toLowerCase();
            word = "";
          }
        }
      }
      // last word
      if (!word.equals("")) {
        words[index++] = word.toLowerCase();
      }
      fr.close();
      // Step 2: Find unique words
      System.out.println("Unique words:");
      for (int i = 0; i < index; i++) {
        int count = 0;
        for (int j = 0; j < index; j++) {
          if (words[i].equals(words[j])) {
            count++;
          }
        }
        if (count == 1) {
          System.out.println(words[i]);
        }
      }
    } catch (IOException e) {
      System.out.println("Error: " + e.getMessage());
    }
  }
}
```

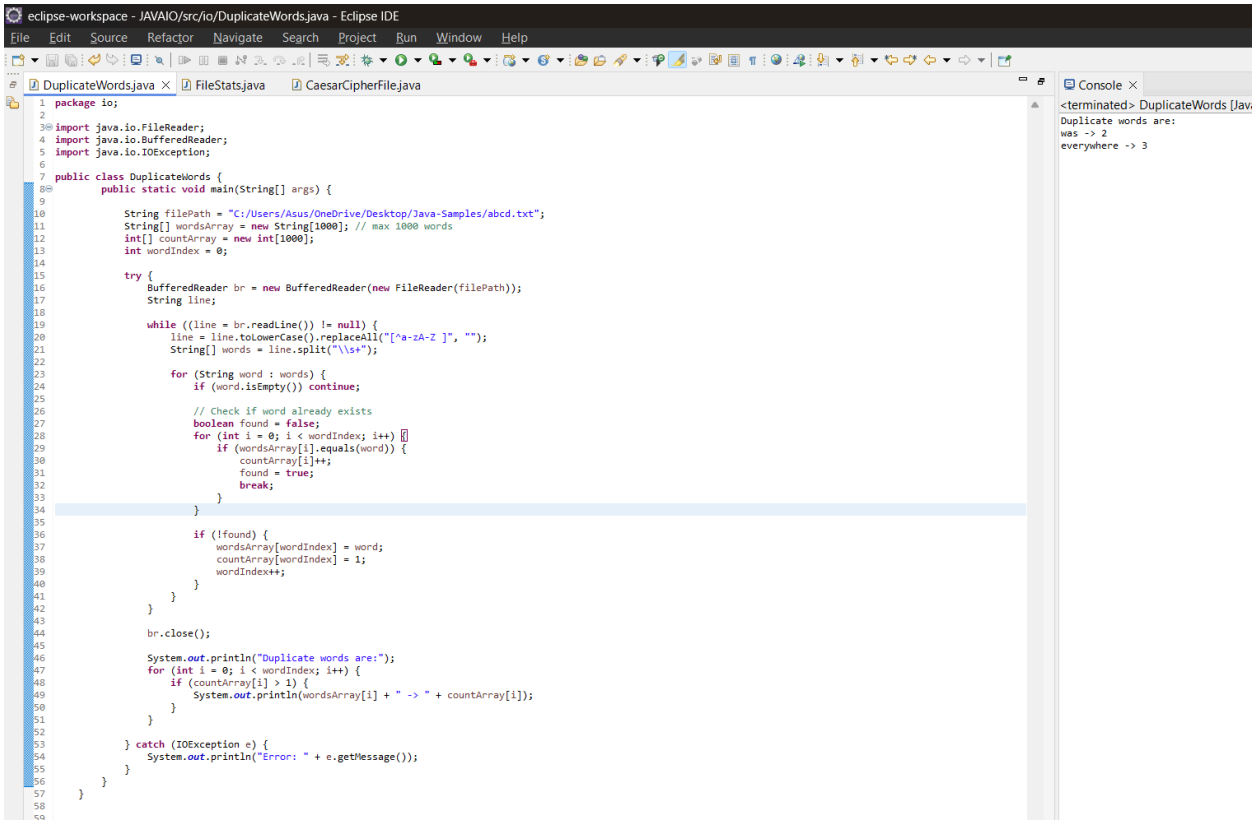


7. Write a program to find duplicate words in a file

Solution:

```
package io;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
public class DuplicateWords {
        public static void main(String[] args) {
            String filePath = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/abcd.txt";
            String[] wordsArray = new String[1000]; // max 1000 words
            int[] countArray = new int[1000];
            int wordIndex = 0;
            try {
                BufferedReader br = new BufferedReader(new FileReader(filePath));
                String line;
                while ((line = br.readLine()) != null) {
                    line = line.toLowerCase().replaceAll("[^a-zA-Z ]", "");
                    String[] words = line.split("\\s+");
```

```
            for (String word : words) {
                if (word.isEmpty()) continue;
                // Check if word already exists
                boolean found = false;
                for (int i = 0; i < wordIndex; i++) {
                    if (wordsArray[i].equals(word)) {
                        countArray[i]++;
                        found = true;
                        break;
                    }
                }
                if (!found) {
                    wordsArray[wordIndex] = word;
                    countArray[wordIndex] = 1;
                    wordIndex++;
                }
            }
        }
        br.close();
        System.out.println("Duplicate words are:");
        for (int i = 0; i < wordIndex; i++) {
            if (countArray[i] > 1) {
                System.out.println(wordsArray[i] + " -> " + countArray[i]);
            }
        }
    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
  }
}
```




```
package io;

import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class DuplicateWords {
        public static void main(String[] args) {

            String filePath = "C:/Users/Asus/OneDrive/Desktop/Java-Samples/abcd.txt";
            String[] wordsArray = new String[1000]; // max 1000 words
            int[] countArray = new int[1000];
            int wordIndex = 0;

            try {
                BufferedReader br = new BufferedReader(new FileReader(filePath));
                String line;

                while ((line = br.readLine()) != null) {
                    line = line.toLowerCase().replaceAll("[^a-zA-Z ]", "");
                    String[] words = line.split("\\s+");

                    for (String word : words) {
                        if (word.isEmpty()) continue;

                        // Check if word already exists
                        boolean found = false;
                        for (int i = 0; i < wordIndex; i++) {
                            if (wordsArray[i].equals(word)) {
                                countArray[i]++;
                                found = true;
                                break;
                            }
                        }

                        if (!found) {
                            wordsArray[wordIndex] = word;
                            countArray[wordIndex] = 1;
                            wordIndex++;
                        }
                    }
                }

                br.close();

                System.out.println("Duplicate words are:");
                for (int i = 0; i < wordIndex; i++) {
                    if (countArray[i] > 1) {
                        System.out.println(wordsArray[i] + " -> " + countArray[i]);
                    }
                }

            } catch (IOException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
```

Multi-Threading:

1. Write a program to print "Good morning" and "Welcome" continuously on the screen in

Java using threads.

Solution:

//1. Write a program to print "Good morning" and "Welcome" continuously on the screen in Java using threads.

```
package threads;

class GoodMorning extends Thread {

        public void run() {
```

```
                while(true) {
                        System.out.println("Good Morning...");
                try {
                        Thread.sleep(500);
                }catch(Exception e) {
                        System.out.println(e);
                }
        }
}
}
class Welcome extends Thread{
        public void run() {
                        while(true) {
                        System.out.println("Welcome...");


                        try {
                        Thread.sleep(500);
                } catch(Exception e) {
                        System.out.println(e);
                }
                }
        }
}
```

```
class test{
	public static void main(String[] args) {
		GoodMorning t1 = new GoodMorning();
		Welcome w1 = new Welcome();


		t1.start();
		w1.start();
	}
}
```

Help

Circle.java | GoodMorning.java

```java
//1. Write a program to print "Good morning" and "Welcome" continuously on the screen in Java using threads.

package threads;

class GoodMorning extends Thread {
    public void run() {
        while(true) {
            System.out.println("Good Morning...");
        try {
            Thread.sleep(500);
        }catch(Exception e) {
            System.out.println(e);
        }
    }
}

}
class Welcome extends Thread{
    public void run() {
            while(true) {
            System.out.println("Welcome...");

                try {
            Thread.sleep(500);
        } catch(Exception e) {
            System.out.println(e);
        }
        }
    }
}

class test{
    public static void main(String[] args) {
        GoodMorning t1 = new GoodMorning();
        Welcome w1 = new Welcome();

        t1.start();
        w1.start();
    }
}
```

Problems | @ Javadoc | Declaration | Console

test [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.e

```
Good Morning...
Welcome...
Good Morning...
Welcome...
Good Morning...
Welcome...
Welcome...
Good Morning...
Welcome...
Good Morning...
Welcome...
Good Morning...
Welcome...
Good Morning...
Welcome...
```

//2. Demonstrate gerPriority() and setPriority() , currentThread() methods in Java threads.

Solution:

```
package threads;

class MYThread extends Thread{

	public void run() {

		System.out.println(Thread.currentThread().getName() + " has priority: " +
getPriority());

	}



}




class PriorityTest{

	public static void main(String[] args) {

		MYThread t1 = new MYThread();

		MYThread t2 = new MYThread();


		t1.setName("Thread-Low");

		t2.setName("Thread-High");


		t1.setPriority(Thread.MIN_PRIORITY);

		t2.setPriority(Thread.MAX_PRIORITY);


		t1.start();

		t2.start();

	}

}
```

Circle.java | GoodMorning.java | MYThread.java

```java
//2.  Demonstrate gerPriority() and setPriority() , currentThread() methods in Java threads.

package threads;

class MYThread extends Thread{
    public void run() {
        System.out.println(Thread.currentThread().getName() + " has priority: " + getPriority());
    }

}


class PriorityTest{
    public static void main(String[] args) {
        MYThread t1 = new MYThread();
        MYThread t2 = new MYThread();

        t1.setName("Thread-Low");
        t2.setName("Thread-High");

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
    }

}
```

Problems | Javadoc | Declaration | Console

```
<terminated> PriorityTest [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.
Thread-High has priority: 10
Thread-Low has priority: 1
```

3. Write a Java program to implement sleep() and join() methods.

Solution:

```
package threads;

class Task extends Thread {

        public void run() {

                System.out.println(getName() + " has started.");
```

```
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                System.out.println("Interupted...");
            }
        }

}
class TestDemo{
        public static void main(String[] args) {
            Task t1 = new Task();
            Task t2 = new Task();

            t1.setName("Thread-1");
            t2.setName("Thread-2");
            t1.start();



            try {
                System.out.println("Main is now waiting for thread-1 to complete.");
                t1.join();
            }catch(InterruptedException e) {
                System.out.println("Main thread was interrupted while joining");
            }
```

```
            t2.start();

        }

}
```

elp

Circle.java | GoodMorning.java | MYThread.java | Task.java

```java
//3. Write a Java program to implement sleep() and join() methods.
package threads;

class Task extends Thread {
    public void run() {
        System.out.println(getName() + " has started.");

        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("Interupted...");
        }
    }

}
class TestDemo{
    public static void main(String[] args) {
        Task t1 = new Task();
        Task t2 = new Task();

        t1.setName("Thread-1");
        t2.setName("Thread-2");
        t1.start();


        try {
            System.out.println("Main is now waiting for thread-1 to complete.");
            t1.join();
        }catch(InterruptedException e) {
            System.out.println("Main thread was interrupted while joining");
        }
        t2.start();
    }
}
```

Problems | @ Javadoc | Declaration | Console

<terminated> TestDemo [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v

```
Main is now waiting for thread-1 to complete.
Thread-1 has started.
Thread-2 has started.
```

/4. Write a Word Count program and count words from two or more files. Write separate threads to

//count words from a specific file and display word count of each file

Soltuion:

```
package multithread;

import java.io.BufferedReader;

import java.io.FileReader;

public class WordCount extends Thread{

		String filename;


	public WordCount(String filename) {

		this.filename = filename;

	}

	public void run() {

		int count = 0;

		try {

			BufferedReader br = new BufferedReader(new FileReader(filename));

			String line;

			while((line = br.readLine() )!= null) {

				String words[] = line.split("\\s+");

				count += words.length;

			}

			br.close();

			System.out.println(filename + " -> " + count);
```

```
		}catch(Exception e) {

			System.out.println(e);

		}

	}

	public static void main(String[] args) {

		WordCount  wc1 = new
WordCount("C:/Users/DAC-User49/Desktop/Temp/abc.txt");

		WordCount wc2 = new
WordCount("C:/Users/DAC-User49/Desktop/Temp/xyz.txt");

			wc1.start();

			wc2.start();

	}

}
```

```java
//4.  Write a Word Count program and count words from two or more files. Write separate threads to
//count words from a specific file and display word count of each file

package multithread;
import java.io.BufferedReader;
import java.io.FileReader;
public class WordCount extends Thread{
        String filename;

    public WordCount(String filename) {
        this.filename = filename;
    }
    public void run() {
        int count = 0;
        try {
            BufferedReader br = new BufferedReader(new FileReader(filename));
            String line;
            while((line = br.readLine() )!= null) {
                String words[] = line.split("\\s+");
                count += words.length;
            }
            br.close();
            System.out.println(filename + " -> " + count);
        }catch(Exception e) {
            System.out.println(e);
        }
    }
    public static void main(String[] args) {
        WordCount  wc1 = new WordCount("C:/Users/DAC-User49/Desktop/Temp/abc.txt");
        WordCount wc2 = new WordCount("C:/Users/DAC-User49/Desktop/Temp/xyz.txt");
            wc1.start();
            wc2.start();
    }

}
```

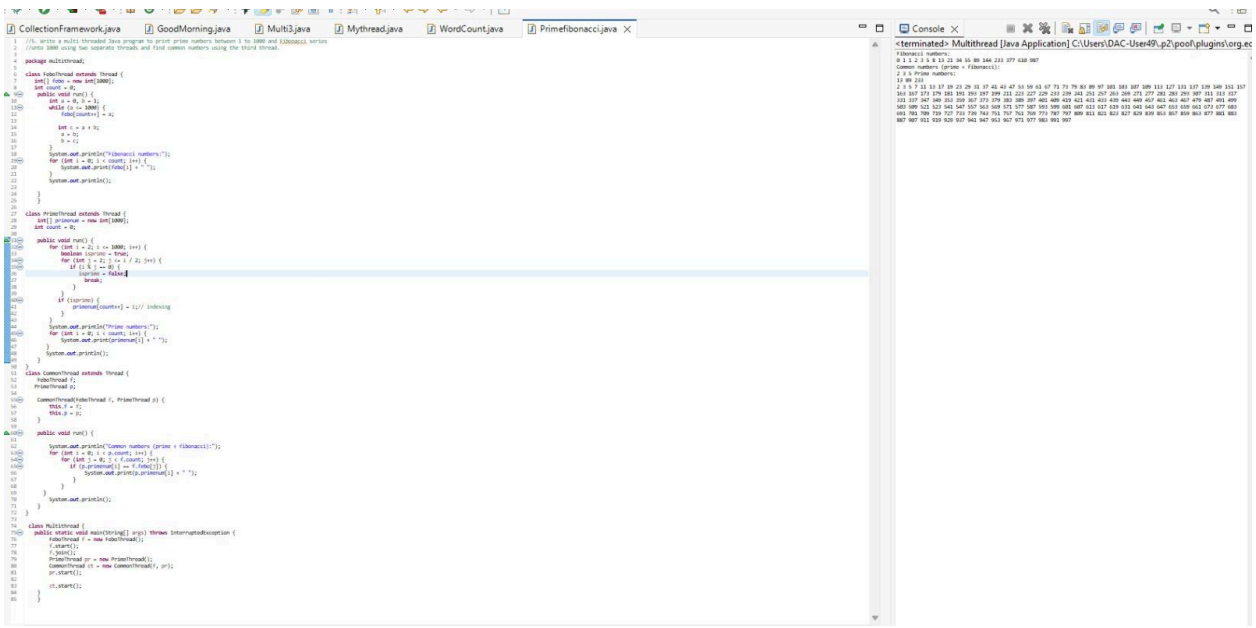
```
Problems  @ Javadoc  Declaration  Console  Coverage
<terminated> WordCount [Java Application] C:\Users\DAC-User49\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\javaw.e
C:/Users/DAC-User49/Desktop/Temp/xyz.txt -> 47
C:/Users/DAC-User49/Desktop/Temp/abc.txt -> 46
```

/5. Write a multi-threaded Java program to print prime numbers between 1 to 1000 and Fibonacci series

//unto 1000 using two separate threads and find common numbers using the third thread.

Solution:

```java
package multithread;

class FeboThread extends Thread {

   int[] febo = new int[1000];

   int count = 0;
```

```
    public void run() {
        int a = 0, b = 1;
        while (a <= 1000) {
            febo[count++] = a;


           int c = a + b;
            a = b;
            b = c;
        }
        System.out.println("Fibonacci numbers:");
        for (int i = 0; i < count; i++) {
            System.out.print(febo[i] + " ");
        }
        System.out.println();


    }
    }


class PrimeThread extends Thread {
    int[] primenum = new int[1000];
   int count = 0;


    public void run() {
        for (int i = 2; i <= 1000; i++) {
```

```
boolean isprime = true;

for (int j = 2; j <= i / 2; j++) {

  if (i % j == 0) {

    isprime = false;

      break;

  }
```